

Advanced Algorithms: Homework 8 Solutions

Due on April 10, 2024 at 11:59pm EST

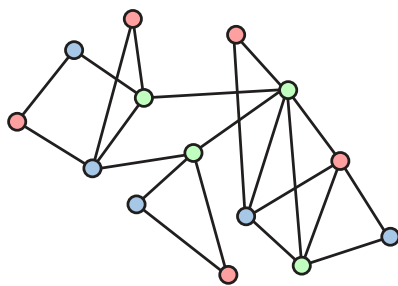
Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use of previous semester course materials is strictly prohibited in this course. EC problems are all-or-nothing. Furthermore, you must complete EC problems alone (i.e. without external resources or collaborators)

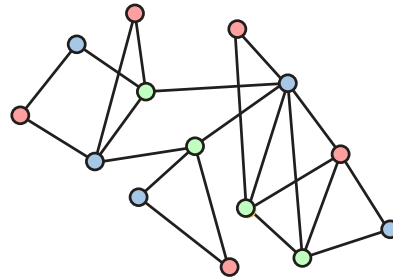
Exercise 1

(Erickson) Let $G = (V, E)$ be an undirected graph, each of whose vertices is colored either **red**, **green**, or **blue**. An edge in G is *boring* if its endpoints have the same color, and *interesting* if its endpoints have different colors. The *most interesting 3-coloring* is the 3-coloring with the maximum number of interesting edges, or equivalently, with the fewest boring edges. Computing the most interesting 3-coloring is NP-hard, because the standard 3-coloring problem is a special case.

- Let $\text{wow}(G)$ denote the number of interesting edges in the most interesting 3-coloring of G . Suppose we assign each vertex in G a random color in $\{\text{red}, \text{green}, \text{blue}\}$. Prove that the expected number of interesting edges is at least $\frac{2}{3}\text{wow}(G)$.
- Prove that with high probability, the expected number of interesting edges is at least $\frac{1}{2}\text{wow}(G)$. (Hint: Use Chebyshev's inequality. But wait... How do we know that we **can** use Chebyshev's inequality?)
- Let $\text{snooze}(G)$ denote the number of boring edges in the most interesting 3-coloring of a graph G . Prove that it is NP-hard to approximate $\text{snooze}(G)$ within a factor of 2024^{2024} .



(a) A somewhat boring 3-coloring.



(b) The most interesting 3-coloring.¹

Figure 1: Here $\text{wow}(G) = 21$ and $\text{snooze}(G) = 1$.

Solution

- When a vertex is randomly colored, each color has a $\frac{1}{3}$ probability of being chosen. There are 3 choices for the color of the first endpoint. For the second endpoint, there are 2 choices (out of 3) that are different from the first endpoint's color. So, the probability that an edge is interesting is $\frac{3 \times 2}{3 \times 3} = \frac{2}{3}$.

By linearity of expectation, the expected number of interesting edges when vertices are randomly colored is $\frac{2}{3}|E| \geq \frac{2}{3}\text{wow}(G)$.

- Chebyshev's inequality states that for any random variable Y with mean μ and variance σ^2 , and for any $k > 0$,

$$\mathbb{P}(|Y - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

To apply Chebyshev's inequality, we need to know the variance of the number of interesting edges. We denote the random variable X_e as 1 if edge e is interesting, and 0 otherwise. Thus, the total number of interesting edges is $X = \sum_{e \in E} X_e$, where each X_e is a Bernoulli variable with probability $\frac{2}{3}$.

The variance of X_e is $\text{Var}(X_e) = p(1 - p) = \frac{2}{3} \times \frac{1}{3} = \frac{2}{9}$.

Although the variables X_e are not mutually independent, they are pairwise independent. Indeed, if two edges are disjoint this is immediate, and if two edges share a vertex, say $e = (u, v)$ and $f = (u, w)$, then

¹Unless I missed something and this graph is actually 3-colorable.

conditioning on the color of u , the events that v and w receive colors different from u are independent, each with probability $2/3$. Thus

$$P(X_e = 1 \text{ and } X_f = 1) = \frac{4}{9} = P(X_e = 1)P(X_f = 1),$$

so $\text{Cov}(X_e, X_f) = 0$.

Using

$$\text{Var}\left(\sum_{e \in E} X_e\right) = \sum_{e \in E} \text{Var}(X_e) + 2 \sum_{e < f} \text{Cov}(X_e, X_f),$$

and since all covariance terms are zero, the total variance is the sum of individual variances:

$$\text{Var}(X) = \sum_{e \in E} \text{Var}(X_e) = \frac{2}{9}|E|.$$

We can apply this to X with $\mu = \frac{2}{3}|E|$ and $\sigma^2 = \frac{2}{9}|E|$, and choose $k = \frac{1}{6}|E|$. Then the probability that X diverges from its mean by more than $k = \frac{1}{6}|E|$ is less than $\frac{8}{|E|}$.

Therefore, with probability at least $1 - \frac{8}{|E|}$,

$$X \geq \frac{1}{2}|E| \geq \frac{1}{2} \text{wow}(G).$$

(c) First, note that computing $\text{snooze}(G)$ is NP-hard by a reduction from 3COLOR, since

$$\exists \text{ 3-coloring of } G \rightarrow \text{snooze}(G) = 0$$

$$\nexists \text{ 3-coloring of } G \rightarrow \text{snooze}(G) \geq 1$$

But now suppose we create an updated graph G' as follows. Let G' be the union of $2024^{2024} + 1$ copies of G , with additional edges added as follows: for every edge uv in G , add an edge from *every* copy of u to *every* copy of v . Then

$$\exists \text{ 3-coloring of } G' \rightarrow \text{snooze}(G') = 0$$

$$\nexists \text{ 3-coloring of } G' \rightarrow \text{snooze}(G') \geq 2024^{2024} + 1$$

Thus, we have a polynomial-time reduction of 3COLOR to an ϵ -approximation of snooze .

Exercise 2²

Define a *triangle matching* in an undirected graph G as a collection of vertex-disjoint triangles (cycles of length 3) in G . A triangle matching is *maximal* if it is not a proper subgraph of a larger triangle matching in the same graph.

- Let M and M' be two arbitrary maximal triangle matchings in the same underlying graph G . Prove that $|M| \leq 3 \cdot |M'|$.
- Finding the *largest* triangle matching in a given graph is NP-hard. Describe and analyze a fast 3-approximation algorithm for this problem.
- Finding the *smallest maximal* triangle matching in a given graph is NP-hard. Describe and analyze a fast 3-approximation algorithm for this problem.

²This could've been an exam problem!

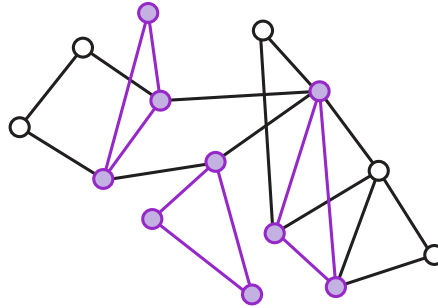


Figure 2: A maximal (but not maximum!) triangle matching

Solution

- (a) Consider an arbitrary triangle Δ in M . If Δ does not share a vertex with any triangle in M' , then $M' + \Delta$ is a triangle matching, contradicting our assumption that M' is maximal. We conclude that every triangle in M shares at least one vertex with a triangle in M' .

On the other hand, consider an arbitrary triangle Δ' in M' . Each vertex of Δ' is a vertex of at most one triangle in M , because the triangles in M are vertex-disjoint. Thus, the number of triangles in M is at most the number of vertices in M' .

- (b) **Solution:** The following brute-force algorithm computes a maximal triangle matching in $O(VE)$ time.

```

unmark every vertex of  $G$ 
 $M \leftarrow \emptyset$ 
for all vertices  $v$  do
  for all edges  $uv$  do
    for all edges  $vw$  do
      if  $u \neq w$  and  $uw \in E$  and  $u, v, w$  are all unmarked then
        mark  $u, v, w$ 
        add  $uvw$  to  $M$ 
return  $M$ 

```

Let M denote the triangle matching computed by our algorithm, and let OPT denote the largest triangle matching in G . Part (a) immediately implies $|\text{OPT}|/|M| \leq 3$.

- (c) Same algorithm as part (b).

Let M denote the triangle matching computed by our algorithm, and let OPT denote the smallest maximal triangle matching in G . Part (a) immediately implies $|M|/|\text{OPT}| \leq 3$.

Exercise 3

Find a polynomial-time $4/3$ -approximation to instances of metric-TSP where distances are either 1 or 2.

Hint: The 2-matching problem (i.e. find a minimum weight subset of edges M such that each node is adjacent to exactly 2 edges in M) can be solved in polynomial time.

Solution

A 2-matching (or more commonly a 2-factor) is a collection of disjoint cycles that spans the graph. The Min-cost 2-matching returns the lowest cost among all 2-matchings. A TSP tour (a 2-matching that contains exactly one cycle), is a type of 2-matching, so $\text{cost}(2\text{-matching}) \leq \text{cost}(\text{OPT})$

Say we run our 2-matching algorithm, and we have k cycles returned. We transform this into a TSP tour as follows: from each cycle i , pick an edge $e_i = (v_{i,1}, v_{i,2})$. We delete these k edges to obtain a collection of paths. We now add edges between these paths to get a final cycle, that is, add edges $(v_{1,2}, v_{2,1}), (v_{2,2}, v_{3,1}), (v_{3,2}, v_{4,1}), \dots (v_{k,2}, v_{1,1})$. This results in a TSP tour.

The total cost is the cost of the 2-matching, minus the cost of the deleted edges, plus the cost of the added edges. The cost of the deleted edges is at least 1 per edge, and the cost of the added edges is at most 2 per edge. Since we add and delete k edges, we increase the cost by at most $2k - k = k$. Since a cycle needs at least 3 vertices, there are at most $n/3$ cycles. Also, since each edge has cost at least 1, $n \leq \text{cost}(OPT)$. Thus,

$$\begin{aligned} \text{cost}(ALG) &= \text{cost}(2\text{-matching}) - \text{cost}(\text{deleted edges}) + \text{cost}(\text{added edges}) \\ &\leq \text{cost}(OPT) + k \leq \text{cost}(OPT) + n/3 \leq \text{cost}(OPT) + \text{cost}(OPT)/3 \\ &= 4/3 \text{cost}(OPT) \end{aligned}$$

Exercise 4 (EC)

You (hopefully) have heard of the **Maximum Independent Set (MIS)** problem from previous courses, where the goal was to find an maximum-cardinality independent set V' of vertices in some graph G . (If not, refer to the [Appendix](#) for more information)

In the **Maximum Independent Set on Unit Disk Graph (MIS_{UDG})** problem, we are given a set S of unit-diameter disks in the plane. The goal is to find a maximum-cardinality subset $S' \subseteq S$ of disks, such that no two disks in S' overlap. (We say that disks that only touch on the boundary do not overlap). As with MIS, this problem is also NP-Complete.

- (8 pts) Consider the following greedy algorithm: start with $S' = \emptyset$ and process the disks in S one-by-one, in any order. For each such disk C , if $S' \cup \{C\}$ is a feasible solution, then add C to S' . What is the approximation factor of this algorithm? (Give the best upper bound you can.)
- (4 pts) Let $0 < \varepsilon < 1$ be some constant. Assume that we are given a grid whose lines are spaced $1/\varepsilon$ units apart. Assume further that no disk in the input set S intersects the grid lines. Give a polynomial time algorithm for solving the problem exactly in this case.
- (8 pts) Derive and analyze a PTAS for the MIS_{UDG} problem.

Hint: Consider a grid with *random offset*: Take a grid of lines spaced $\frac{1}{2}$ apart, such that the origin is at the intersection of a horizontal and vertical grid line. Pick a shift/offset L uniformly at random from $[0, \frac{1}{2})$, and shift the grid vertically and horizontally by an distance L . (Equivalently, consider the grid of spacing $\frac{1}{2}$ such that the point (L, L) is at the intersection of two grid lines.) What is the probability that a disk is intersected by a grid line?

This problem has applications in VLSI chip manufacturing (Again, refer to the [Appendix](#) for more information.)

Solution

- 5-Approximation. Easy to show (think of a hexagonal packing). More specifically, every disk in the optimal solution either appears in the greedy solution or overlaps some disk in the greedy solution. Moreover, any disk in the greedy solution can overlap at most 5 disks in the optimal solution, since those disks must be disjoint unit-diameter disks whose centers lie within distance 1 of its center. Thus $|OPT| \leq 5|ALG|$.
- Consider any grid box containing at least one disk. This grid box is of dimensions $\frac{1}{\varepsilon} \times \frac{1}{\varepsilon}$, so it can contain at most $k = \mathcal{O}(\frac{1}{\varepsilon^2})$ disks such that no two disks overlap. Thus we can solve the MIS_{UDG}

problem when restricted to this grid box simply by considering every subset of disks of size $\leq k$ and checking to see if any of them overlap. The total number of subsets we have to consider is

$$\sum_{i=1}^k \binom{n}{i} = \mathcal{O}(n^k)$$

and the time it takes to check if any of them overlap is also $\mathcal{O}(n^k)$.

There are at most $|S|$ grid boxes containing at least one disk. Since no disk intersects a grid line, disks in different boxes cannot overlap, so we solve each box independently and take the union of the solutions. Thus we have a poly time algorithm.

- (c) Pick a random grid offset with spacing $\frac{1}{\epsilon}$. For any disk, the probability that it is hit by a vertical grid line is at most ϵ , and the probability that it is hit by a horizontal grid line is at most ϵ . Thus, by union bound, each disk is hit by some grid line with probability at most 2ϵ . Remove those circles. Run the algorithm from part (b). In expectation, at least a $(1 - 2\epsilon)$ fraction of the optimal solution remains, so the returned solution has size at least $(1 - 2\epsilon)|OPT|$ in expectation. Replacing ϵ by $\epsilon/2$ gives a $(1 - \epsilon)$ -approximation, and thus a PTAS.

Appendix

Maximum Independent Set

An **independent set** in a graph is a subset of the vertices with no edges between them. Finding the largest independent set in an arbitrary graph is famously **NP-hard**. But in some special classes of graphs, we can find largest independent sets quickly. In particular, when the input graph is a tree with n vertices, we can actually compute the largest independent set in $O(n)$ time using **dynamic programming**, as follows:

For any node v in T , let $MIS(v)$ denote the size of the largest independent set in the subtree rooted at v . Any independent set in this subtree that excludes v itself is the union of independent sets in the subtrees rooted at the children of v . On the other hand, any independent set that includes v necessarily excludes all of v 's children, and therefore includes independent sets in the subtrees rooted at v 's grandchildren. Thus, the table MIS obeys the following recurrence, which we can compute using a post-order traversal on the tree (here the set $C(v)$ contains the children of v):

$$MIS(v) = \max \left\{ \sum_{w \in C(v)} MIS(w), 1 + \sum_{w \in C(v)} \sum_{x \in C(w)} MIS(x) \right\}$$

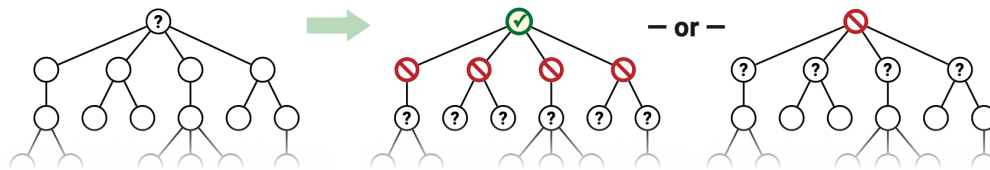


Figure 3: Computing maximum independent set in a tree.

Configurability in VLSI Chip Manufacturing³

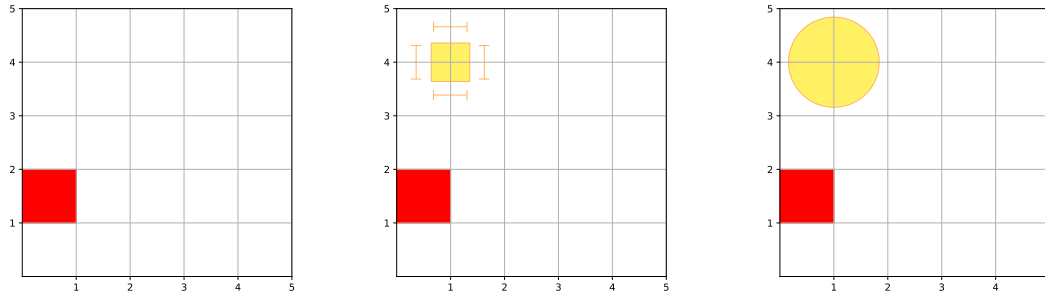
We begin with a motivating question: Why is it so hard to create large chips? One reason is because of precision. It is very hard to avoid defects in the creation of these chips. Thus, a construct called **configurability** was introduced to VLSI chip design.

In manufacturing, chips are regions of a larger unit called a “wafer.” Ordinarily after fabrication, a wafer is tested and faulty chips are marked; then the wafer is diced, the faulty chips are discarded, and the functional chips are packaged. However, since the entire chip is being manufactured at once, this is wasteful because even if only a small portion of the chip is defective, it must be entirely discarded.

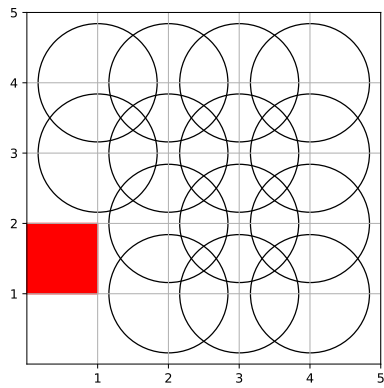
To use configurability, this process is changed somewhat. Special circuitry is added that enables adjacent chips to be connected. Then, after the wafer is tested, adjacent functional chips are connected into a group and the entire group is used as a single enlarged chip. In this way, even if a smaller chips becomes defective, the adjacent chips can be rewired so that they can still be used in the larger chip.

A natural problem that arises from this setup is packing a maximal number of working chips on a wafer. As an example, consider the Cartesian grid representing a wafer, shown below. Wherever there is a 2×2 grid of non-defective chips, these grid cells can be wired together. This potential grouping of chips can be visualized as a unit disk which overlaps with the four grid cells that are being wired together. For example, in the figure below, the four grid cells in the upper left corner could be wired together, so we would put a unit disk at $(1, 4)$ with radius 1.

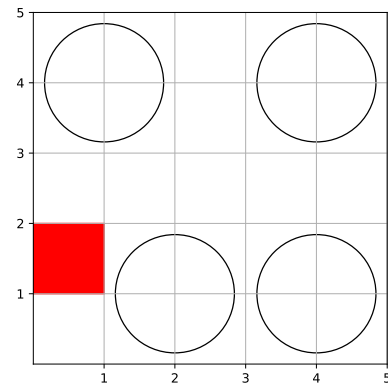
³I have limited knowledge in this field, so forgive me if I abuse some terminology.



Continuing this process for every possible wiring, we obtain the collection of disks shown in Figure 4a. Overlapping unit disks represents two wirings which conflict with each other because they use the same chips. In this scenario, the MIS_{UDG} represents the maximum number of non-conflicting wirings, as shown in Figure 4b.



(a) Unit disks representing chip wirings.



(b) The MIS_{UDG} of unit disks.